

```
manchodu@Srini:~$ cd ~/OSLAB
manchodu@Srini:~/OSLAB$ nano tokenizer.c
manchodu@Srini:~/OSLAB$ gcc tokenizer.c -o tokenizer
manchodu@Srini:~/OSLAB$ ./tokenizer
Simple Command Tokenizer
Enter a command:
> ls -l /home

--- Token Stream ---
Token 0: WORD           Value = 'ls'
Token 1: WORD           Value = '-l'
Token 2: WORD           Value = '/home'

--- Validation ---
Token stream is valid.
manchodu@Srini:~/OSLAB$ ./tokenizer
Simple Command Tokenizer
Enter a command:
> ls || grep

--- Token Stream ---
Token 0: WORD           Value = 'ls'
Token 1: PIPE           Value = '|'
Token 2: PIPE           Value = '|'
Token 3: WORD           Value = 'grep'

--- Validation ---
Syntax Error: Consecutive pipes are not allowed.
Token stream is invalid.
```

```

    }
}

/* Add a token */
void add_token(Token tokens[], int *count, TokenType type, const char *value)
{
    if (*count >= MAX_TOKENS)
    {
        printf("Error: Too many tokens!\n");
        return;
    }

    tokens[*count].type = type;

    strncpy(tokens[*count].value, value, MAX_TOKEN_LENGTH - 1);
    tokens[*count].value[MAX_TOKEN_LENGTH - 1] = '\0';

    (*count)++;
}

/* Tokenize input */
int tokenize(char *input, Token tokens[])
{
    int count = 0;
    int i = 0;

    while (input[i] != '\0')
    {
        /* Ignore whitespace */
        if (isspace(input[i]))
        {
            i++;
            continue;
        }

        /* Pipe */
        if (input[i] == '|')
        {
            add_token(tokens, &count, TOKEN_PIPE, "|");
            i++;
            continue;
        }

        /* Input redirection */
        if (input[i] == '<')
        {
            add_token(tokens, &count, TOKEN_REDIRECT_IN, "<");

```

```

        add_token(tokens, &count, TOKEN_REDIRECT_IN, "<");
        i++;
        continue;
    }

    /* Output redirection */
    if (input[i] == '>')
    {
        add_token(tokens, &count, TOKEN_REDIRECT_OUT, ">");
        i++;
        continue;
    }

    /* Background */
    if (input[i] == '&')
    {
        add_token(tokens, &count, TOKEN_BACKGROUND, "&");
        i++;
        continue;
    }

    /* Normal word */
    if (!isspace(input[i]))
    {
        char word[MAX_TOKEN_LENGTH];
        int j = 0;

        while (input[i] != '\0' &&
                !isspace(input[i]) &&
                input[i] != '|' &&
                input[i] != '<' &&
                input[i] != '>' &&
                input[i] != '&')
        {
            if (j < MAX_TOKEN_LENGTH - 1)
            {
                word[j++] = input[i];
            }

            i++;
        }

        word[j] = '\0';

        add_token(tokens, &count, TOKEN_WORD, word);
    }
}

```

```

/* Validate token stream */
int validate_tokens(Token tokens[], int count)
{
    if (count == 0)
    {
        printf("Empty command.\n");
        return 0;
    }

    /* Command cannot start with pipe */
    if (tokens[0].type == TOKEN_PIPE)
    {
        printf("Syntax Error: Command cannot start with '|'.\n");
        return 0;
    }

    /* Command cannot end with pipe */
    if (tokens[count - 1].type == TOKEN_PIPE)
    {
        printf("Syntax Error: Command cannot end with '|'.\n");
        return 0;
    }

    /* Two pipes cannot occur together */
    for (int i = 0; i < count - 1; i++)
    {
        if (tokens[i].type == TOKEN_PIPE &&
            tokens[i + 1].type == TOKEN_PIPE)
        {
            printf("Syntax Error: Consecutive pipes are not allowed.\n");
            return 0;
        }
    }

    return 1;
}

int main()
{
    char input[MAX_INPUT];
    Token tokens[MAX_TOKENS];

    printf("Simple Command Tokenizer\n");
    printf("Enter a command:\n");

```

```

Token tokens[MAX_TOKENS];

printf("Simple Command Tokenizer\n");
printf("Enter a command:\n");

printf("> ");

if (fgets(input, sizeof(input), stdin) == NULL)
{
    return 1;
}

int count = tokenize(input, tokens);

printf("\n--- Token Stream ---\n");

if (count == 0)
{
    printf("No tokens found.\n");
    return 0;
}

for (int i = 0; i < count; i++)
{
    printf("Token %d: %-15s Value = '%s'\n",
           i,
           token_type_name(tokens[i].type),
           tokens[i].value);
}

printf("\n--- Validation ---\n");

if (validate_tokens(tokens, count))
{
    printf("Token stream is valid.\n");
}
else
{
    printf("Token stream is invalid.\n");
}

return 0;
}

```